

ALLIANCE UNIVERSITY

Private University established in Karnataka State by Act No. 34 of year 2010
Recognised by the University Grants Commission (UGC), New Delhi

DATABASE MANAGEMENT SYSTEM ASSIGNMENT-8

CAB BOOKING MANAGEMENT SYSTEM

Basic SQL Queries and Aggregate Functions

BACHELORS OF COMPUTER APPLICATION- III

NAME: PREKSHA V

REGISTER NUMBER: 2511021240063

SUBMITTING TO,

RENITA JOY R

1. Objective

Thisassignment demonstrates SQL querying on the Cab Booking database. The work covers row filtering with WHERE, sorting with ORDER BY, grouping with GROUP BY, group filtering with HAVING, aggregate functions including COUNT, SUM, AVG, MIN and MAX, and joins/subqueries across related tables. The actual query and output screenshot supplied for each exercise is included below.

2. Queries and Results

EachsectionexplainsthepurposeoftheSQL operation and reproduces the corresponding execution screenshot.

Query 1: Filtering completed bookings

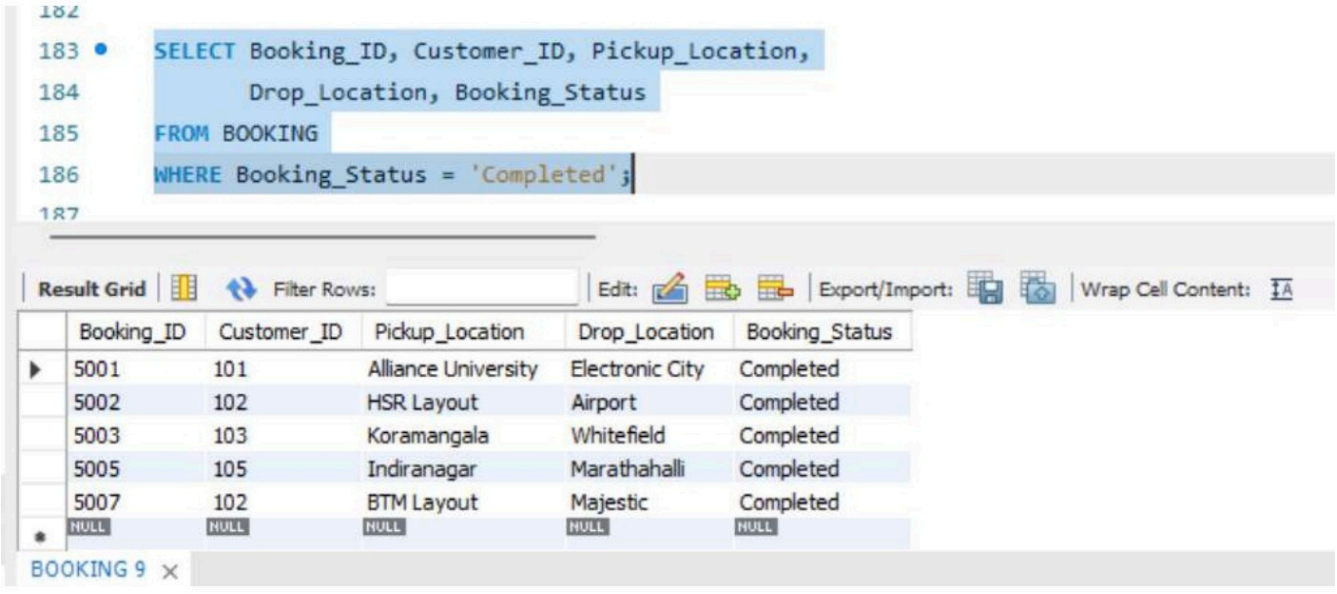
SQL QUERY

```
SELECT Booking_ID, Customer_ID, Pickup_Location,
     Drop_Location, Booking_Status

FROM BOOKING
WHERE Booking_Status = 'Completed';
```

Explanation: Filters the BOOKING table to show only records whose status is Completed. The result contains five completed bookings.

EXECUTION SCREENSHOT



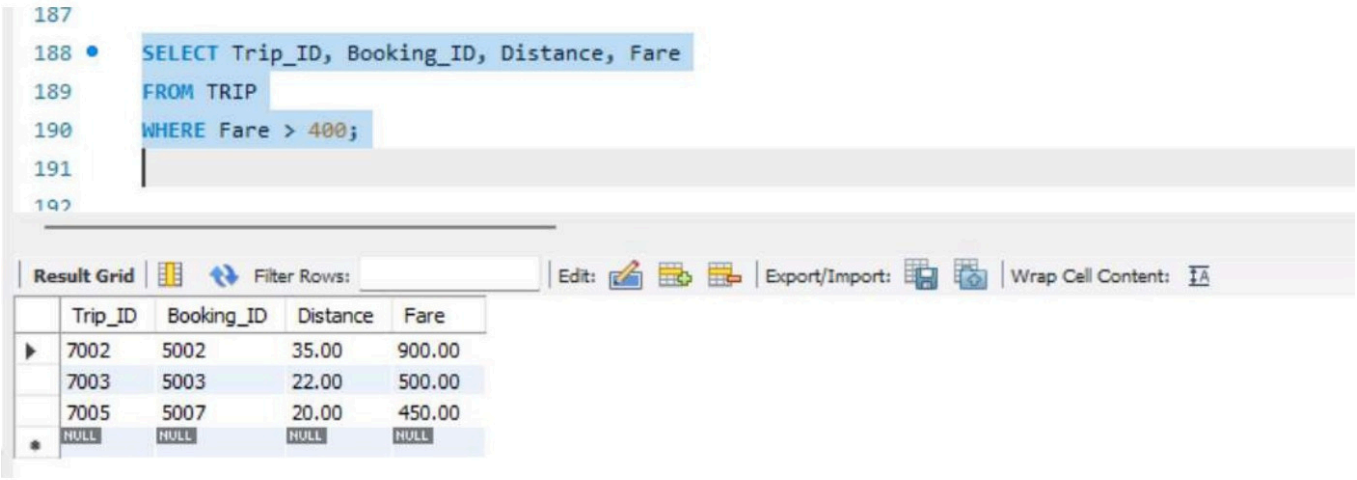
Query 2: Filtering trips by fare

SQL QUERY

```
SELECT Trip_ID, Booking_ID, Distance, Fare
FROM TRIP
WHERE Fare > 400;
```

Explanation: The WHERE condition returns trips with a fare greater than 400. Three trips match this threshold.

EXECUTION SCREENSHOT



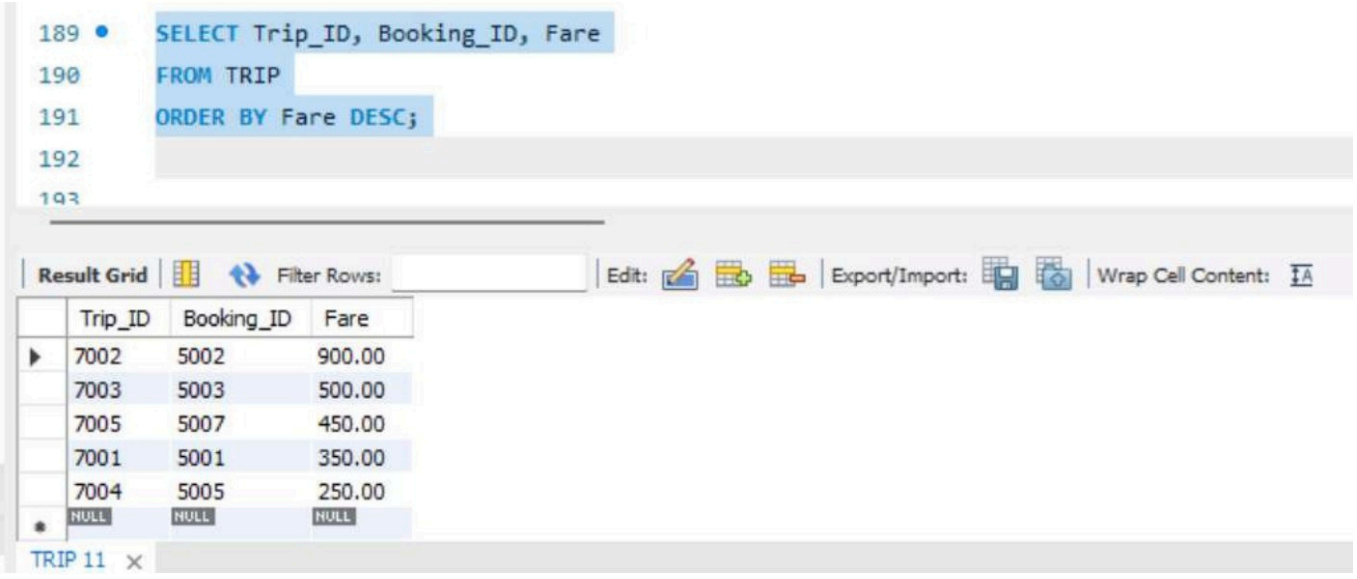
Query 3: Sorting trips by fare

SQL QUERY

```
SELECT Trip_ID, Booking_ID, Fare
FROM TRIP
ORDER BY Fare DESC;
```

Explanation: ORDER BY with DESC ranks the trip fares from highest to lowest, making the most expensive trip appear first.

EXECUTION SCREENSHOT



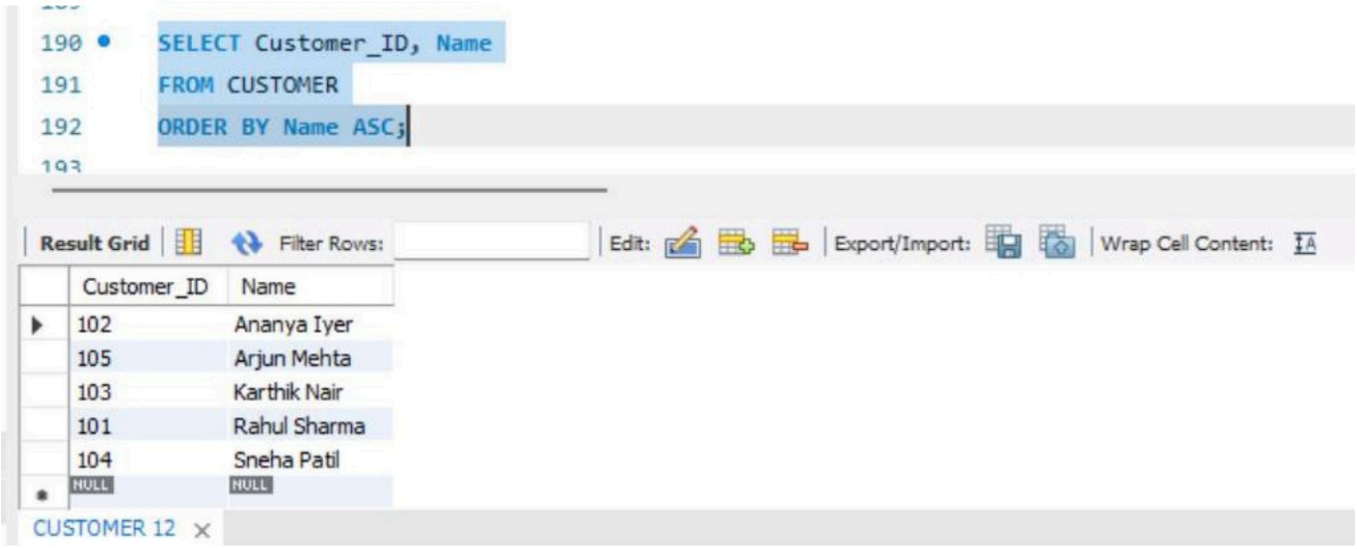
Query 4: Sorting customers alphabetically

SQL QUERY

```
SELECT Customer_ID, Name
FROM CUSTOMER
ORDER BY Name ASC;
```

Explanation: ASC sorts customer names alphabetically. The query returns the five customers in the database.

EXECUTION SCREENSHOT



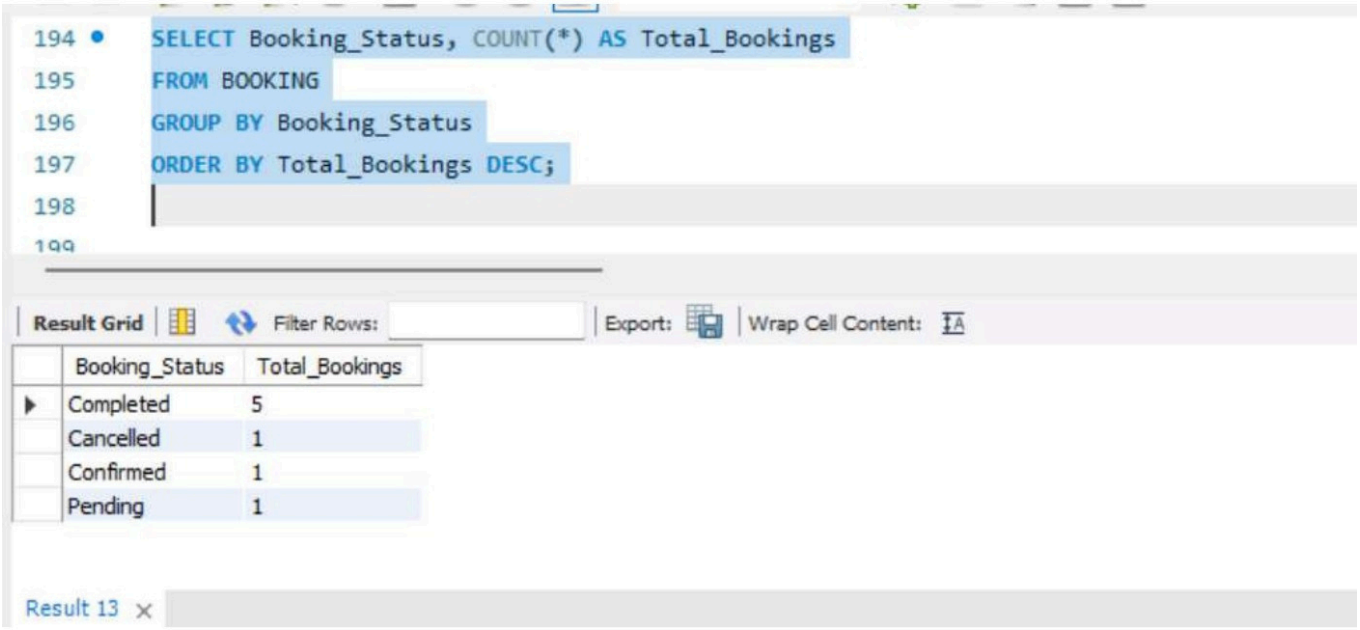
Query 5: Counting bookings by status

SQL QUERY

```
SELECT Booking_Status, COUNT(*) AS Total_Bookings
                        FROM BOOKING
                        GROUP BY Booking_Status
                        ORDER BY Total_Bookings DESC;
```

Explanation: GROUP BY produces one row per booking status, and COUNT(*) counts bookings in each group. The output shows five Completed and one each of Cancelled, Confirmed, and Pending.

EXECUTION SCREENSHOT



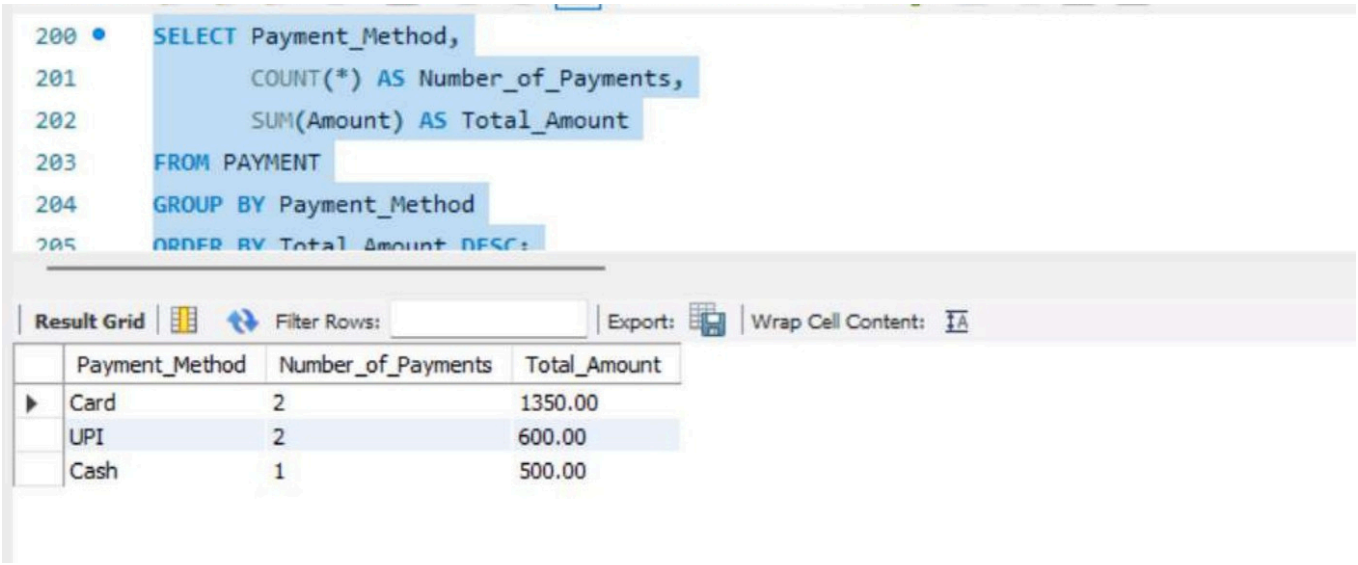
Query 6: Payment totals by payment method

SQL QUERY

```
SELECT Payment_Method, COUNT(*) AS Number_of_Payments,
     SUM(Amount) AS Total_Amount
 FROM PAYMENT
 GROUP BY Payment_Method
 ORDER BY Total_Amount DESC;
```

Explanation: This groups payment records by method and calculates both the number and summed amount for each method. Card totals 1,350, UPI 600, and Cash 500.

EXECUTION SCREENSHOT



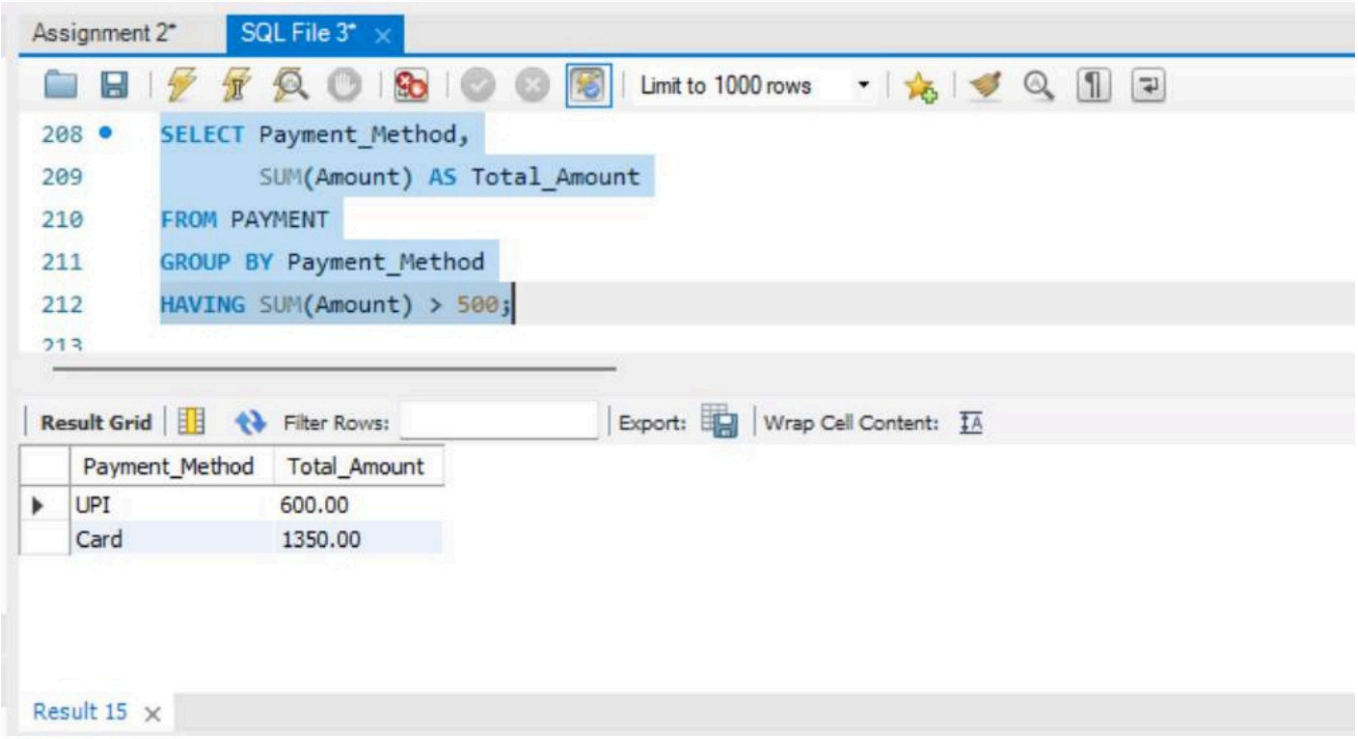
Query 7: Filtering payment groups with HAVING

SQL QUERY

```
SELECT Payment_Method, SUM(Amount) AS Total_Amount
FROM PAYMENT
GROUP BY Payment_Method
HAVING SUM(Amount) > 500;
```

Explanation: HAVING filters the grouped results after aggregation, retaining only methods with a total payment amount above 500. UPI and Card qualify.

EXECUTION SCREENSHOT



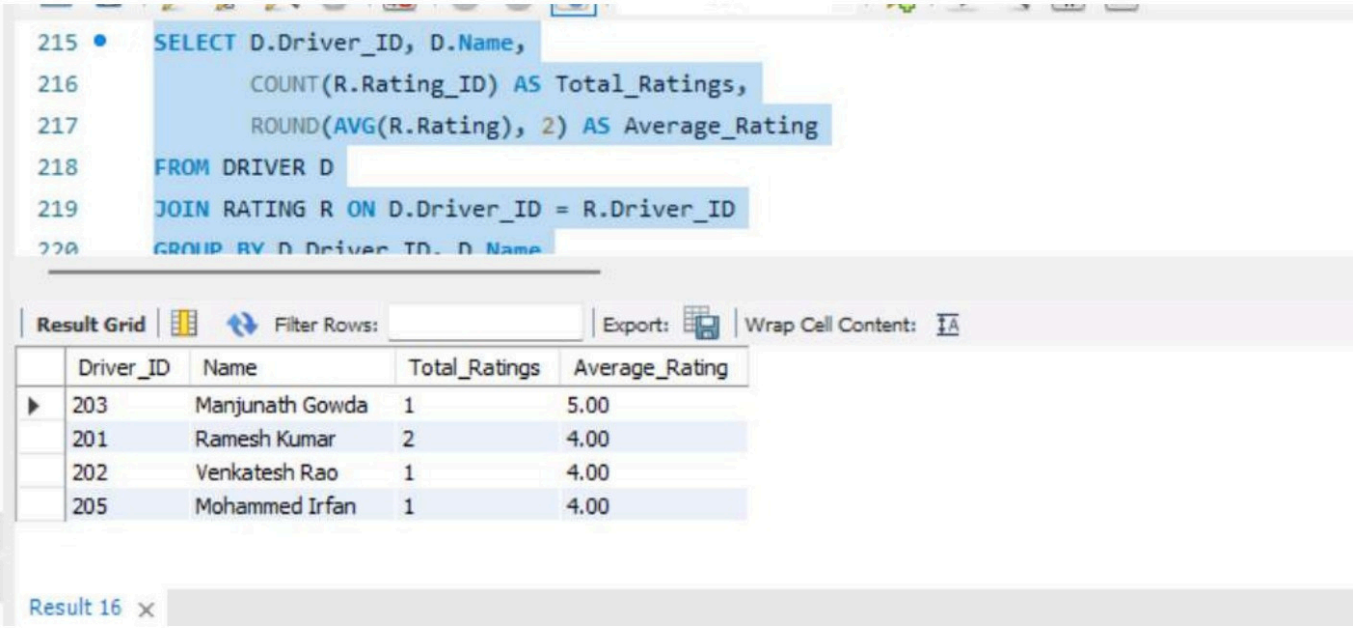
Query 8: Average driver rating

SQL QUERY

```
SELECT D.Driver_ID, D.Name,
       COUNT(R.Rating_ID) AS Total_Ratings,
       ROUND(AVG(R.Rating), 2) AS Average_Rating
FROM DRIVER D
JOIN RATING R ON D.Driver_ID = R.Driver_ID
GROUP BY D.Driver_ID, D.Name;
```

Explanation: The join associates ratings with drivers. COUNT and AVG summarize each driver's reviews; ROUND limits the average to two decimal places.

EXECUTION SCREENSHOT



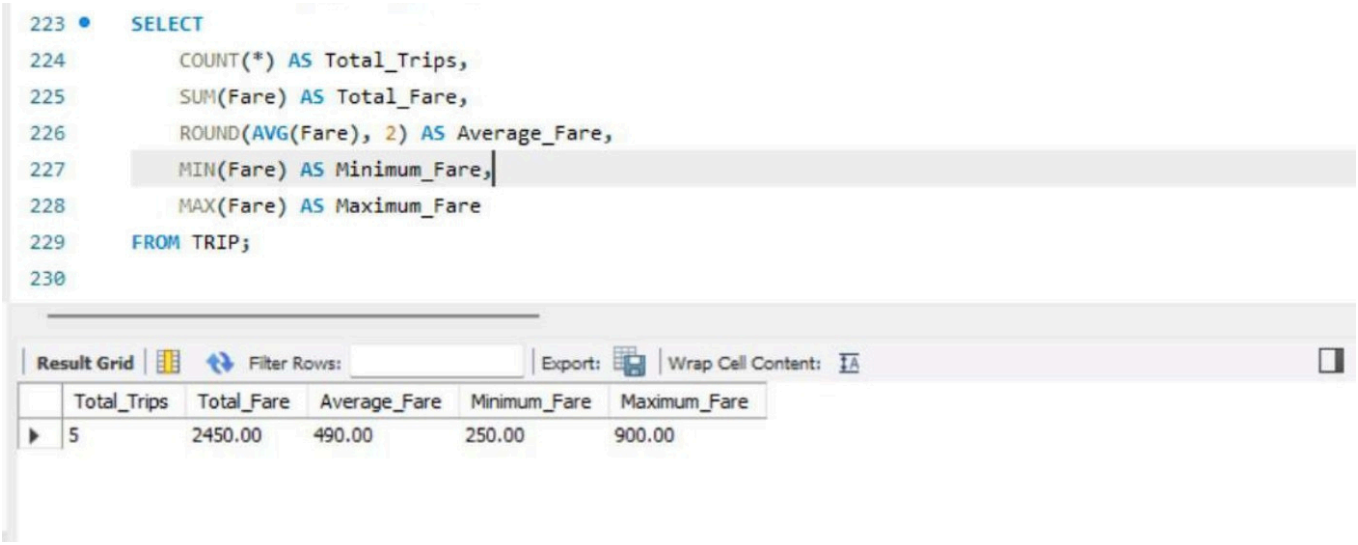
Query 9: Summary statistics for trip fares

SQL QUERY

```
SELECT COUNT(*) AS Total_Trips,
       SUM(Fare) AS Total_Fare,
       ROUND(AVG(Fare), 2) AS Average_Fare,
       MIN(Fare) AS Minimum_Fare,
       MAX(Fare) AS Maximum_Fare
FROM TRIP;
```

Explanation: This aggregate query returns a single summary row: five trips, total fares of 2,450, average fare 490, minimum 250, and maximum 900.

EXECUTION SCREENSHOT



Query 10: Summary statistics for ratings

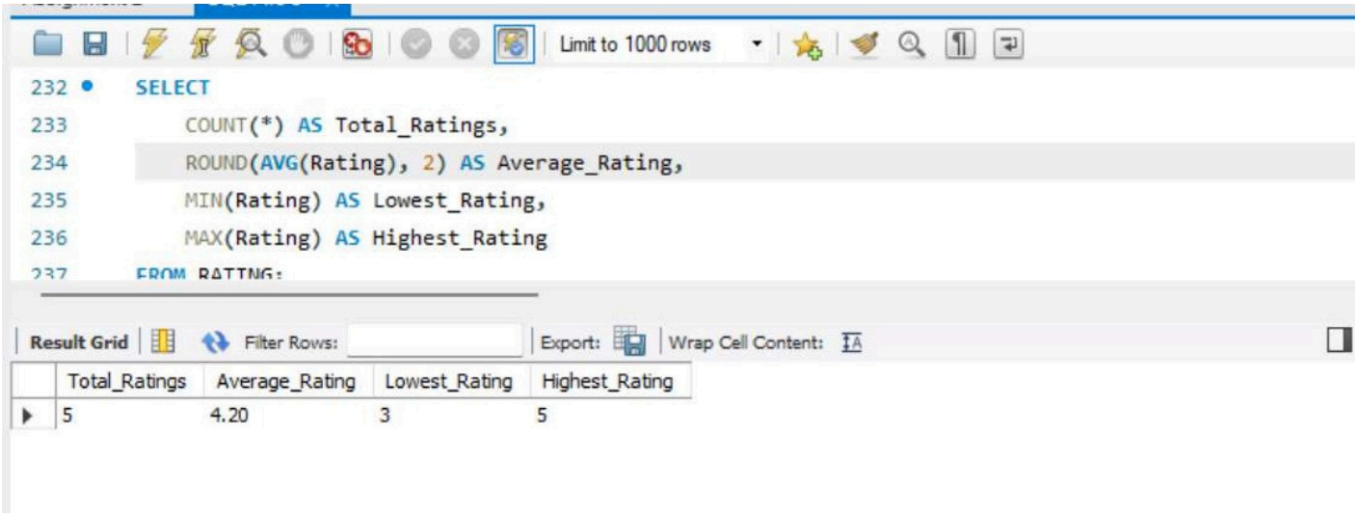
SQL QUERY

```
SELECT COUNT(*) AS Total_Ratings,
   ROUND(AVG(Rating), 2) AS Average_Rating,
   MIN(Rating)       AS       Lowest_Rating,
   MAX(Rating) AS Highest_Rating

FROM RATING;
```

Explanation: COUNT, AVG, MIN, and MAX summarize all rating records. The result shows five ratings, a 4.20 average, and a range from 3 to 5.

EXECUTION SCREENSHOT



Query 11: Customers with more than one booking

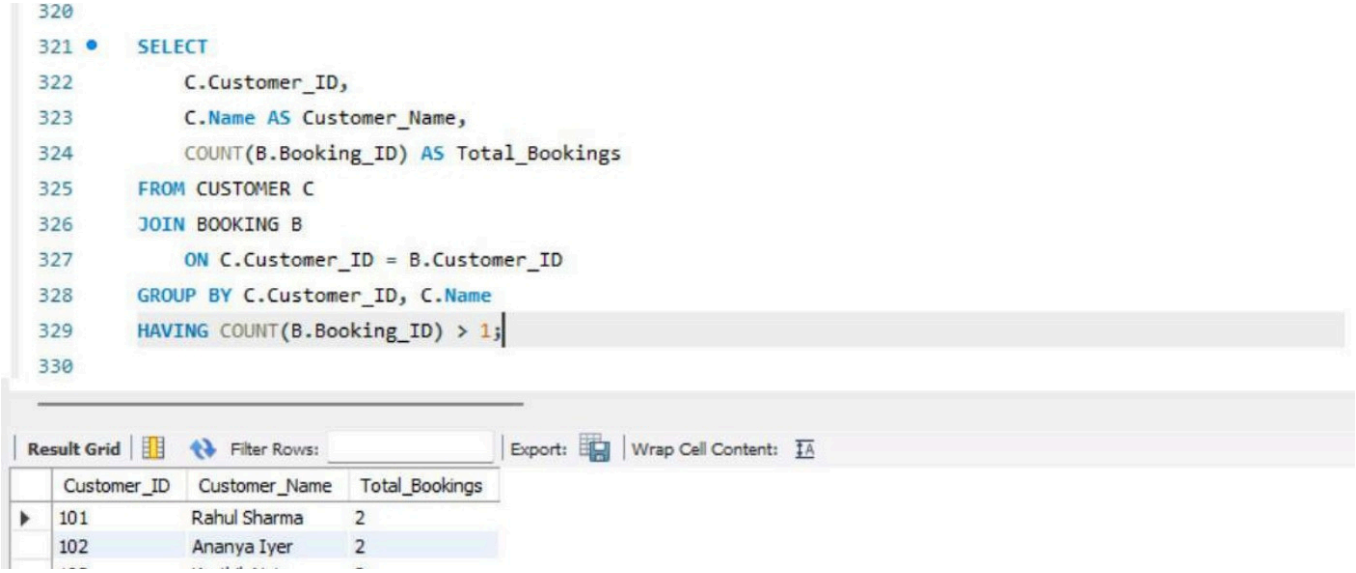
SQL QUERY

```
SELECT C.Customer_ID, C.Name AS Customer_Name,
     COUNT(B.Booking_ID) AS Total_Bookings

 FROM CUSTOMER C
 JOIN BOOKING B ON C.Customer_ID = B.Customer_ID
 GROUP BY C.Customer_ID, C.Name
 HAVING COUNT(B.Booking_ID) > 1;
```

Explanation: The join connects customers to their bookings. GROUP BY calculates each customer's count, while HAVING keeps customers with more than one booking. Three customers are returned.

EXECUTION SCREENSHOT



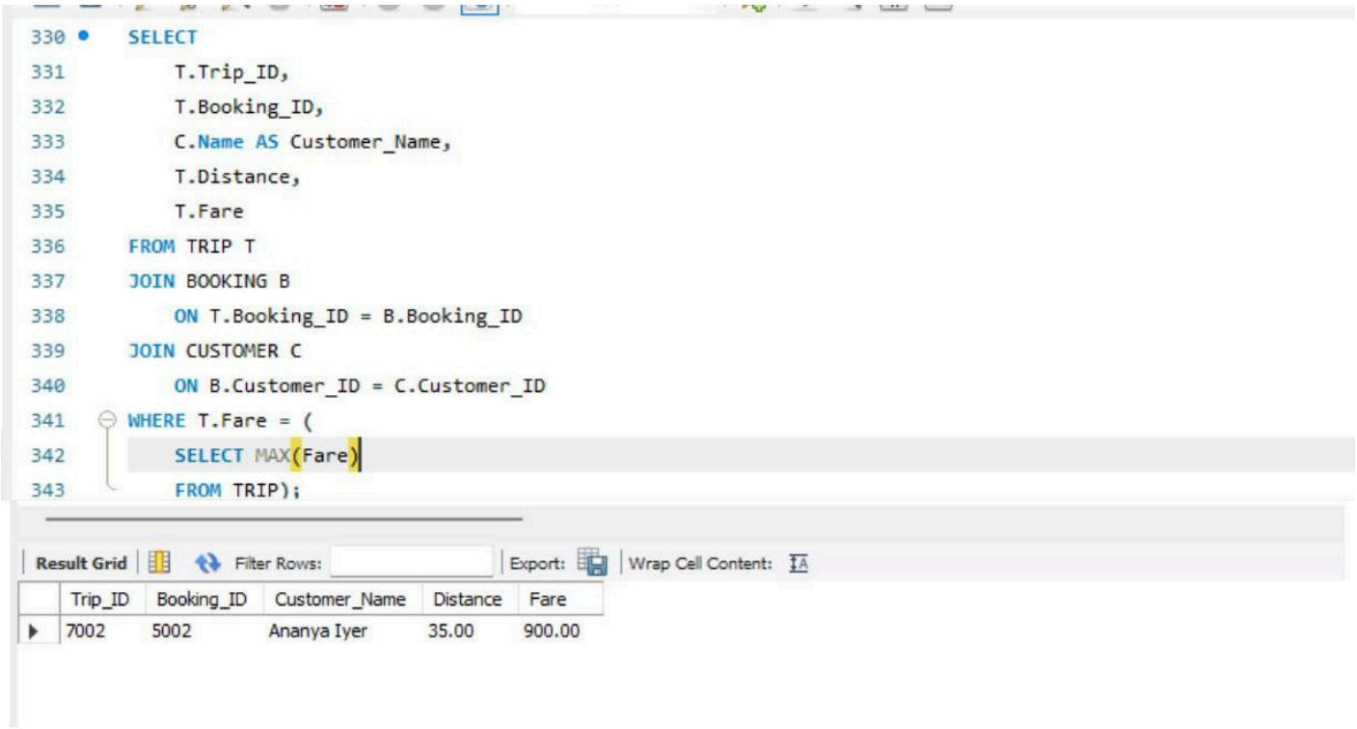
Query 12: Trip with the maximum fare

SQL QUERY

```
SELECT T.Trip_ID, T.Booking_ID, C.Name AS Customer_Name,
       T.Distance, T.Fare
FROM TRIP T
JOIN BOOKING B ON T.Booking_ID = B.Booking_ID
JOIN CUSTOMER C ON B.Customer_ID = C.Customer_ID
WHERE T.Fare = (SELECT MAX(Fare) FROM TRIP);
```

Explanation: A scalar subquery finds the maximum fare; the outer query retrieves that trip and joins booking/customer details. Trip 702 has the maximum fare of 900.

EXECUTION SCREENSHOT



Query 13: Booking counts by driver

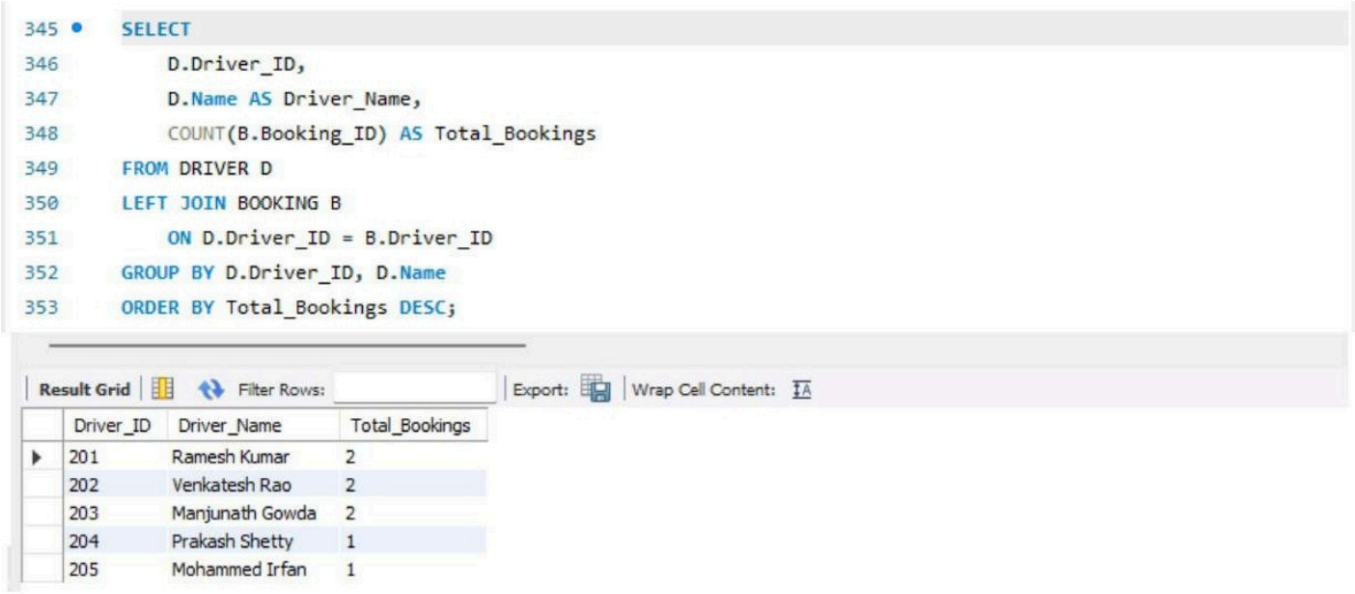
SQL QUERY

```
SELECT D.Driver_ID, D.Name AS Driver_Name,
       COUNT(B.Booking_ID) AS Total_Bookings

FROM DRIVER D
LEFT JOIN BOOKING B ON D.Driver_ID = B.Driver_ID
GROUP BY D.Driver_ID, D.Name
ORDER BY Total_Bookings DESC;
```

Explanation: LEFT JOIN preserves drivers even if they have no matching booking. The grouping counts each driver's bookings, sorted from most to fewest.

EXECUTION SCREENSHOT



```
345 •  SELECT
346        D.Driver_ID,
347        D.Name AS Driver_Name,
348        COUNT(B.Booking_ID) AS Total_Bookings
349    FROM DRIVER D
350    LEFT JOIN BOOKING B
351        ON D.Driver_ID = B.Driver_ID
352    GROUP BY D.Driver_ID, D.Name
353    ORDER BY Total_Bookings DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Driver_ID	Driver_Name	Total_Bookings
201	Ramesh Kumar	2
202	Venkatesh Rao	2
203	Manjunath Gowda	2
204	Prakash Shetty	1
205	Mohammed Irfan	1

Query 14: Combined booking, trip, and payment details

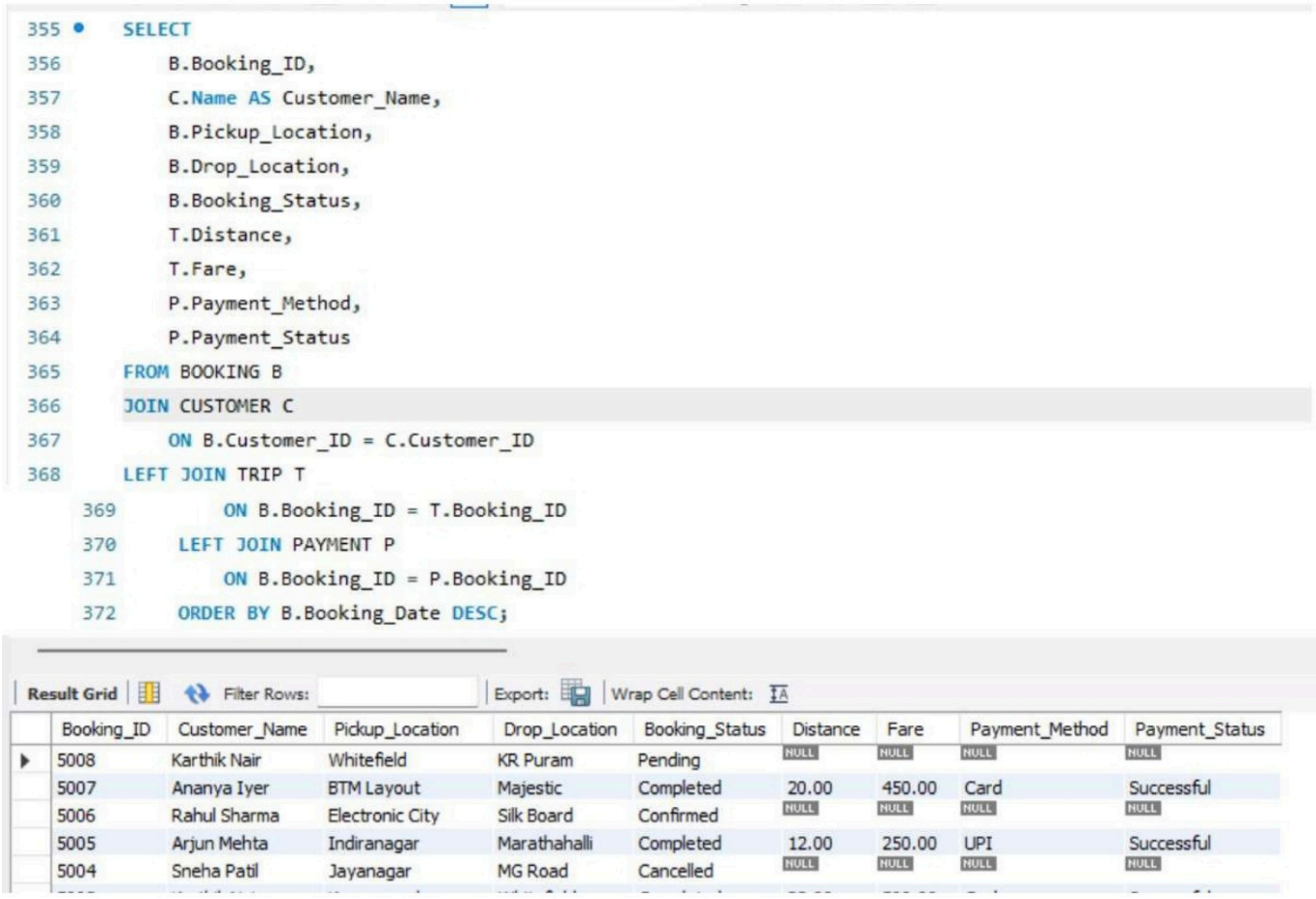
SQL QUERY

```
SELECT B.Booking_ID, C.Name AS Customer_Name,
       B.Pickup_Location, B.Drop_Location, B.Booking_Status,
       T.Distance, T.Fare, P.Payment_Method, P.Payment_Status

FROM BOOKING B
JOIN CUSTOMER C ON B.Customer_ID = C.Customer_ID
LEFT JOIN TRIP T ON B.Booking_ID = T.Booking_ID
LEFT JOIN PAYMENT P ON B.Booking_ID = P.Booking_ID
ORDER BY B.Booking_Date DESC;
```

Explanation: This query combines bookings with customer names and optional trip/payment details. LEFT JOIN keeps bookings even when a trip or payment record is missing; results are newest first.

EXECUTION SCREENSHOT



```
355 •  SELECT
356        B.Booking_ID,
357        C.Name AS Customer_Name,
358        B.Pickup_Location,
359        B.Drop_Location,
360        B.Booking_Status,
361        T.Distance,
362        T.Fare,
363        P.Payment_Method,
364        P.Payment_Status
365    FROM BOOKING B
366    JOIN CUSTOMER C
367        ON B.Customer_ID = C.Customer_ID
368    LEFT JOIN TRIP T
369        ON B.Booking_ID = T.Booking_ID
370    LEFT JOIN PAYMENT P
371        ON B.Booking_ID = P.Booking_ID
372    ORDER BY B.Booking_Date DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Booking_ID	Customer_Name	Pickup_Location	Drop_Location	Booking_Status	Distance	Fare	Payment_Method	Payment_Status
5008	Karthik Nair	Whitefield	KR Puram	Pending	NULL	NULL	NULL	NULL
5007	Ananya Iyer	BTM Layout	Majestic	Completed	20.00	450.00	Card	Successful
5006	Rahul Sharma	Electronic City	Silk Board	Confirmed	NULL	NULL	NULL	NULL
5005	Arjun Mehta	Indiranagar	Marathahalli	Completed	12.00	250.00	UPI	Successful
5004	Sneha Patil	Jayanagar	MG Road	Cancelled	NULL	NULL	NULL	NULL

3. Database Tables and Sample Records

Thefinalscreenshotprovidesan overview ofthecab-bookingdatabase tables and sample rows used in the assignment. The displayed data connects customers and bookings with drivers, vehicles, trips, and payments.

	Customer_ID	Customer_Name	Customer_Phone	Booking_ID	Booking_Date	Pickup_Location	Drop_Location	Booking_Status
1	101	Rahul Sharma	9876500001	5001	2026-10-20 09:00:00	Alliance University	Electronic City	Completed
2	102	Ananya Iyer	9876500002	5002	2026-10-20 10:00:00	HSR Layout	Airport	Completed
3	103	Karthik Nair	9876500003	5003	2026-10-21 11:00:00	Koramangala	Whitefield	Completed
4	104	Sneha Patil	9876500004	5004	2026-10-21 12:00:00	Jayanagar	MG Road	Cancelled
5	105	Arjun Mehta	9876500005	5005	2026-10-22 13:00:00	Indiranagar	Marathahalli	Completed
6	101	Rahul Sharma	9876500001	5006	2026-10-22 14:00:00	Electronic City	Silk Board	Confirmed
7	102	Ananya Iyer	9876500002	5007	2026-10-23 09:00:00	BTM Layout	Majestic	Completed
8	103	Karthik Nair	9876500003	5008	2026-10-23 10:00:00	Whitefield	KR Puram	Pending

	Driver_ID	Driver_Name	Driver_Status	Cab_ID	Vehicle_Number	Cab_Type	Model	Trip_ID
1	201	Ramesh Kumar	Available	301	KA01AB1231	Sedan	Honda City	7001
2	202	Venkatesh Rao	Busy	302	KA01AB1232	SUV	Hyundai Creta	7002
3	203	Manjunath Gowda	Available	303	KA01AB1233	Hatchback	Swift	7003
4	204	Prakash Shetty	Offline	304	KA01AB1234	Sedan	Honda Amaze	NULL
5	205	Mohammed Irfan	Available	305	KA01AB1235	SUV	Kia Seltos	7004
6	202	Venkatesh Rao	Busy	302	KA01AB1232	SUV	Hyundai Creta	NULL
7	201	Ramesh Kumar	Available	301	KA01AB1231	Sedan	Honda City	7005
8	203	Manjunath Gowda	Available	303	KA01AB1233	Hatchback	Swift	NULL

	Distance	Fare	Trip_Status	Payment_ID	Payment_Amount	Payment_Method	Payment_Status	Rating
1	18.50	350.00	Completed	8001	350.00	UPI	Successful	5
2	35.00	900.00	Completed	8002	900.00	Card	Successful	4
3	22.00	500.00	Completed	8003	500.00	Cash	Successful	5
4	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
5	12.00	250.00	Completed	8004	250.00	UPI	Successful	4
6	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
7	20.00	450.00	Completed	8005	450.00	Card	Successful	3
8	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

4. Conclusion

The queries show how SQL can retrieve targeted booking records, organize and summarize data, filter grouped results, and combine information from related tables. Aggregates provide concise operational summaries such as payment totals, trip fares, booking counts, and driver ratings.